部门管理模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 部门实体表(department)

Department类：对应部门表，包含部门基本信息属性和关联关系
【springboot/src/main/java/org/example/springboot/entity/Department.java】

```
@Data
@TableName("department")
@Schema(description = "科室实体类")
public class Department {
    @TableId(type = IdType.AUTO)
    @Schema(description = "科室ID")
    private Long id;

    @Schema(description = "科室名称")
    private String deptName;

    @Schema(description = "科室编码")
    private String deptCode;

    @Schema(description = "科室描述")
    private String description;

    @Schema(description = "负责人ID")
    private Long directorId;

    @Schema(description = "状态(0:禁用,1:启用)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "负责人")
    private Doctor director;
}
```

1.2 核心属性说明

- **id**: 科室ID，主键，自增
- **deptName**: 科室名称，如内科、外科等
- **deptCode**: 科室编码，唯一标识符
- **description**: 科室描述，详细介绍科室信息

半夜撕代码版权所有

- **directorId**: 科室负责人ID，关联医生表
- **status**: 科室状态，0表示禁用，1表示启用
- **createTime**: 创建时间
- **updateTime**: 更新时间
- **director**: 科室负责人信息，非数据库字段，用于前端展示

二、各功能详细讲解

2.1 科室列表查询功能

2.1.1 功能概述

科室列表查询功能允许管理员查看系统中所有科室信息，支持按科室名称、科室编码和状态进行筛选，并提供分页展示。查询结果包括科室的基本信息及其负责人详情。

2.1.2 详细讲解功能实现流程

1. 前端在 `vue3/src/views/backend/department/index.vue` 文件中初始化时调用 `fetchDepartments()` 方法
2. 该方法通过 `request.get('/department/page')` 向后端发送GET请求，传递查询参数
3. 后端 `DepartmentController` 的 `getDepartmentsByPage` 方法接收请求
4. 控制器调用 `departmentService.getDepartmentsByPage` 方法处理业务逻辑
5. 服务层方法构建查询条件，查询数据库并返回分页结果
6. 服务层还会查询每个科室的负责人信息，丰富返回结果
7. 前端接收数据并在表格中展示

2.1.3 关键代码讲解

1. 前端查询实现
   【vue3/src/views/backend/department/index.vue: fetchDepartments()】

```
const fetchDepartments = async () => {
  loading.value = true
  try {
    await request.get('/department/page', {
      deptName: searchForm.deptName,
      deptCode: searchForm.deptCode,
      status: searchForm.status,
      currentPage: currentPage.value,
      size: pageSize.value
    }, {
      onSuccess: (res) => {
        tableData.value = res.records
        total.value = res.total
      }
    })
  } catch (error) {
    console.error('获取科室列表失败:', error)
  } finally {
    loading.value = false
```

半夜撕代码版权所有

```
    }
}
```

2. 后端控制器实现
   【springboot/src/main/java/org/example/springboot/controller/DepartmentController.java: getDepartmentsByPage】

```
@Operation(summary = "分页查询科室列表")
@GetMapping("/page")
public Result<?> getDepartmentsByPage(
        @RequestParam(required = false) String deptName,
        @RequestParam(required = false) String deptCode,
        @RequestParam(required = false) Integer status,
        @RequestParam(defaultValue = "1") Integer currentPage,
        @RequestParam(defaultValue = "10") Integer size) {
    return Result.success(departmentService.getDepartmentsByPage(deptName,
deptCode, status, currentPage, size));
}
```

3. 后端服务层实现
   【springboot/src/main/java/org/example/springboot/service/DepartmentService.java: getDepartmentsByPage】

```
public Page<Department> getDepartmentsByPage(String deptName, String deptCode,
Integer status, Integer currentPage, Integer size) {
    LambdaQueryWrapper<Department> queryWrapper = new LambdaQueryWrapper<>();

    // 添加查询条件
    if (StringUtils.isNotBlank(deptName)) {
        queryWrapper.like(Department::getDeptName, deptName);
    }
    if (StringUtils.isNotBlank(deptCode)) {
        queryWrapper.like(Department::getDeptCode, deptCode);
    }
    if (status != null) {
        queryWrapper.eq(Department::getStatus, status);
    }

    // 按科室编码排序
    queryWrapper.orderByAsc(Department::getDeptCode);

    Page<Department> page = departmentMapper.selectPage(new Page<>(currentPage,
size), queryWrapper);

    // 查询负责人信息
    for (Department department : page.getRecords()) {
        if (department.getDirectorId() != null) {
            Doctor director =
doctorMapper.selectById(department.getDirectorId());
            department.setDirector(director);
        }
    }

    return page;
```

```
}
```

2.1.4 功能实现细节

1. 使用MyBatis-Plus的`LambdaQueryWrapper`构建动态查询条件，支持模糊查询和精确匹配
2. 查询结果按科室编码升序排序，便于用户查找
3. 对查询结果进行二次处理，填充科室负责人信息
4. 前端支持按科室名称、科室编码和状态进行筛选，实现多条件查询
5. 使用Element Plus的表格组件展示数据，支持分页功能

2.2 科室新增功能

2.2.1 功能概述

科室新增功能允许管理员创建新的科室信息，包括科室名称、科室编码、科室描述、负责人和状态等信息。系统会验证科室编码的唯一性，确保数据的有效性。

2.2.2 详细讲解功能实现流程

1. 前端点击"新增科室"按钮，调用`handleAdd()`方法，打开新增对话框
2. 填写表单后，点击"确定"按钮，调用`submitForm()`方法
3. 表单验证通过后，通过`request.post('/department', departmentForm)`向后端发送POST请求
4. 后端`DepartmentController`的`createDepartment`方法接收请求
5. 控制器调用`departmentService.createDepartment`方法处理业务逻辑
6. 服务层方法先检查科室编码是否已存在，然后设置创建时间，最后保存数据
7. 前端接收成功响应后，关闭对话框并刷新科室列表

2.2.3 关键代码讲解

1. 前端新增实现
   【vue3/src/views/backend/department/index.vue: submitForm()】

```
const submitForm = () => {
  departmentFormRef.value.validate(async (valid) => {
    if (valid) {
      if (dialogType.value === 'add') {
        // 新增科室
        await request.post('/department', departmentForm, {
          successMsg: '新增科室成功',
          onSuccess: () => {
            dialogVisible.value = false
            fetchDepartments()
          }
        })
      } else {
        // 编辑科室
        // ...省略编辑代码
      }
    }
  })
```

半夜撕代码版权所有

```
}
```

2. 后端控制器实现
   【springboot/src/main/java/org/example/springboot/controller/DepartmentController.java: createDepartment】

```
@Operation(summary = "新增科室")
@PostMapping
public Result<?> createDepartment(@RequestBody Department department) {
    LOGGER.info("新增科室: {}", department);
    Department newDepartment = departmentService.createDepartment(department);
    return Result.success(newDepartment);
}
```

3. 后端服务层实现
   【springboot/src/main/java/org/example/springboot/service/DepartmentService.java: createDepartment】

```
@Transactional
public Department createDepartment(Department department) {
    // 检查科室编码是否已存在
    if (departmentMapper.selectOne(
            new LambdaQueryWrapper<Department>()
                .eq(Department::getDeptCode, department.getDeptCode())
        ) != null) {
        throw new ServiceException("科室编码已存在");
    }

    // 设置创建时间
    department.setCreateTime(LocalDateTime.now());
    department.setUpdateTime(LocalDateTime.now());

    // 如果设置了负责人，检查负责人是否存在
    if (department.getDirectorId() != null) {
        Doctor doctor = doctorMapper.selectById(department.getDirectorId());
        if (doctor == null) {
            throw new ServiceException("负责人不存在");
        }
    }

    if (departmentMapper.insert(department) <= 0) {
        throw new ServiceException("科室添加失败");
    }

    return department;
}
```

2.2.4 功能实现细节

1. 表单验证确保必填字段不为空，提高数据质量
2. 科室编码唯一性检查，避免数据冲突
3. 负责人存在性校验，确保数据一致性

半夜撕代码版权所有

4. 使用 `@Transactional` 注解确保事务完整性
5. 自动设置创建时间和更新时间
6. 前端使用Element Plus的表单组件和对话框组件，提供良好的用户体验

2.3 科室编辑功能

2.3.1 功能概述

科室编辑功能允许管理员修改已有科室的信息，包括科室名称、科室编码、科室描述、负责人和状态等。系统会验证科室编码的唯一性（排除自身），确保数据的有效性。

2.3.2 详细讲解功能实现流程

1. 前端点击科室列表中的"编辑"按钮，调用 `handleEdit(row)` 方法，打开编辑对话框
2. 表单预填充当前科室信息
3. 修改表单后，点击"确定"按钮，调用 `submitForm()` 方法
4. 表单验证通过后，通过 `request.put('/department/${departmentForm.id}', departmentForm)` 向后端发送PUT请求
5. 后端 `DepartmentController` 的 `updateDepartment` 方法接收请求
6. 控制器调用 `departmentService.updateDepartment` 方法处理业务逻辑
7. 服务层方法先检查科室是否存在，再检查科室编码是否与其他科室冲突，然后更新数据
8. 前端接收成功响应后，关闭对话框并刷新科室列表

2.3.3 关键代码讲解

1. 前端编辑实现
   【vue3/src/views/backend/department/index.vue: handleEdit()和submitForm()】

```
// 编辑科室
const handleEdit = (row) => {
  dialogType.value = 'edit'
  Object.keys(departmentForm).forEach(key => {
    departmentForm[key] = row[key]
  })
  dialogVisible.value = true
}

// 提交表单（编辑部分）
const submitForm = () => {
  departmentFormRef.value.validate(async (valid) => {
    if (valid) {
      if (dialogType.value === 'add') {
        // ...省略新增代码
      } else {
        // 编辑科室
        await request.put(`/department/${departmentForm.id}`, departmentForm, {
          successMsg: '编辑科室成功',
          onSuccess: () => {
            dialogVisible.value = false
            fetchDepartments()
          }
```

```
        })
      }
    }
  })
}
```

2. 后端控制器实现
   【springboot/src/main/java/org/example/springboot/controller/DepartmentController.java: updateDepartment】

```
@Operation(summary = "更新科室信息")
@PutMapping("/{id}")
public Result<?> updateDepartment(@PathVariable Long id, @RequestBody Department department) {
    LOGGER.info("更新科室信息: id={}, department={}", id, department);
    departmentService.updateDepartment(id, department);
    return Result.success();
}
```

3. 后端服务层实现
   【springboot/src/main/java/org/example/springboot/service/DepartmentService.java: updateDepartment】

```
public void updateDepartment(Long id, Department department) {
    // 检查科室是否存在
    if (departmentMapper.selectById(id) == null) {
        throw new ServiceException("科室不存在");
    }

    // 如果修改了科室编码，检查是否与其他科室冲突
    if (StringUtils.isNotBlank(department.getDeptCode())) {
        Department existDept = departmentMapper.selectOne(
            new LambdaQueryWrapper<Department>()
                .eq(Department::getDeptCode, department.getDeptCode())
        );
        if (existDept != null && !existDept.getId().equals(id)) {
            throw new ServiceException("科室编码已被其他科室使用");
        }
    }

    // 如果设置了负责人，检查负责人是否存在
    if (department.getDirectorId() != null) {
        Doctor doctor = doctorMapper.selectById(department.getDirectorId());
        if (doctor == null) {
            throw new ServiceException("负责人不存在");
        }
    }

    department.setId(id);
    department.setUpdateTime(LocalDateTime.now());

    if (departmentMapper.updateById(department) <= 0) {
        throw new ServiceException("科室信息更新失败");
    }
```

半夜撕代码版权所有

```
}
```

2.3.4 功能实现细节

1. 编辑前检查科室是否存在，避免操作不存在的数据
2. 科室编码唯一性检查时排除自身ID，允许保持原编码不变
3. 负责人存在性校验，确保数据一致性
4. 自动更新修改时间
5. 前端表单预填充当前科室信息，提高用户体验

2.4 科室删除功能

2.4.1 功能概述

科室删除功能允许管理员删除不再需要的科室。系统会检查科室下是否有医生，如有则禁止删除，确保数据完整性。

2.4.2 详细讲解功能实现流程

1. 前端点击科室列表中的"删除"按钮，弹出确认对话框
2. 确认后调用 `handleDelete(row)` 方法
3. 通过 `request.delete('/department/${row.id}')` 向后端发送DELETE请求
4. 后端 `DepartmentController` 的 `deleteDepartment` 方法接收请求
5. 控制器调用 `departmentService.deleteDepartment` 方法处理业务逻辑
6. 服务层方法先检查科室是否存在，再检查科室下是否有医生，最后执行删除操作
7. 前端接收成功响应后，刷新科室列表

2.4.3 关键代码讲解

1. 前端删除实现
   【vue3/src/views/backend/department/index.vue: handleDelete()】

```
const handleDelete = async (row) => {
  try {
    await request.delete(`/department/${row.id}`, {
      successMsg: '删除成功',
      onSuccess: () => {
        fetchDepartments()
      }
    })
  } catch (error) {
    console.error('删除科室失败:', error)
  }
}
```

2. 后端控制器实现
   【springboot/src/main/java/org/example/springboot/controller/DepartmentController.java: deleteDepartment】

半夜撕代码版权所有

```java
@Operation(summary = "删除科室")
@DeleteMapping("/{id}")
public Result<?> deleteDepartment(@PathVariable Long id) {
    LOGGER.info("删除科室: id={}", id);
    departmentService.deleteDepartment(id);
    return Result.success();
}
```

3. 后端服务层实现
   【springboot/src/main/java/org/example/springboot/service/DepartmentService.java: deleteDepartment】

```java
@Transactional
public void deleteDepartment(Long id) {
    // 检查科室是否存在
    if (departmentMapper.selectById(id) == null) {
        throw new ServiceException("科室不存在");
    }

    // 检查科室下是否有医生
    long doctorCount = doctorMapper.selectCount(
        new LambdaQueryWrapper<Doctor>()
            .eq(Doctor::getDepartmentId, id)
    );
    if (doctorCount > 0) {
        throw new ServiceException("该科室下有医生，无法删除");
    }

    if (departmentMapper.deleteById(id) <= 0) {
        throw new ServiceException("科室删除失败");
    }
}
```

2.4.4 功能实现细节

1. 删除前检查科室是否存在，避免操作不存在的数据
2. 检查科室下是否有医生，防止误删除有关联数据的科室
3. 使用 `@Transactional` 注解确保事务完整性
4. 前端使用确认对话框，防止误操作
5. 删除成功后自动刷新列表，保持数据同步

2.5 前台科室展示功能

2.5.1 功能概述

前台科室展示功能为患者提供医院科室信息浏览，包括科室介绍、负责人信息和科室医生列表。患者可以通过此功能了解各科室的专业特色，并查看医生信息，为后续就诊提供参考。

半夜撕代码版权所有

2.5.2 详细讲解功能实现流程

1. 前端在 `vue3/src/views/frontend/department/index.vue` 文件中初始化时调用 `fetchDepartments()` 方法
2. 该方法通过 `request.get('/department/list')` 获取所有科室列表
3. 用户点击左侧科室菜单，触发 `handleSelect(index)` 方法
4. 该方法通过 `fetchDepartmentDetail(index)` 获取科室详情
5. 同时调用 `fetchDepartmentDoctors(index)` 获取科室下的医生列表
6. 前端根据返回数据展示科室详情和医生信息

2.5.3 关键代码讲解

1. 前端获取科室列表
   【vue3/src/views/frontend/department/index.vue: fetchDepartments()】

```
const fetchDepartments = async () => {
  try {
    await request.get('/department/list', {}, {
      onSuccess: (res) => {
        departments.value = res || []
        if (departments.value.length > 0) {
          activeIndex.value = departments.value[0].id.toString()
          handleSelect(activeIndex.value)
        }
      }
    })
  } catch (error) {
    console.error('获取科室列表失败:', error)
  }
}
```

2. 前端获取科室详情
   【vue3/src/views/frontend/department/index.vue: fetchDepartmentDetail()】

```
const fetchDepartmentDetail = async (id) => {
  try {
    await request.get(`/department/${id}`, {}, {
      onSuccess: async (res) => {
        currentDepartment.value = res

        // 如果有科室主任，获取主任信息
        if (currentDepartment.value.directorId) {
          await fetchDoctorDetail(currentDepartment.value.directorId)
        }

        // 获取科室医生
        await fetchDepartmentDoctors(id)
      }
    })
  } catch (error) {
    console.error('获取科室详情失败:', error)
  }
```

半夜撕代码版权所有

```
}
```

3. 前端获取科室医生列表
   【vue3/src/views/frontend/department/index.vue: fetchDepartmentDoctors()】

```
const fetchDepartmentDoctors = async (deptId) => {
  try {
    await request.get(`/doctor/department/${deptId}`, {}, {
      onSuccess: (res) => {
        doctors.value = res || []
      }
    })
  } catch (error) {
    console.error('获取科室医生列表失败:', error)
  }
}
```

4. 后端获取科室列表实现
   【springboot/src/main/java/org/example/springboot/service/DepartmentService.java: getAllDepartments】

```
public List<Department> getAllDepartments() {
    List<Department> departments = departmentMapper.selectList(
        new LambdaQueryWrapper<Department>()
            .orderByAsc(Department::getDeptCode)
    );

    // 查询负责人信息
    for (Department department : departments) {
        if (department.getDirectorId() != null) {
            Doctor director =
doctorMapper.selectById(department.getDirectorId());
            department.setDirector(director);
        }
    }

    return departments;
}
```

2.5.4 功能实现细节

1. 前台页面采用左右布局，左侧为科室列表，右侧为科室详情
2. 科室详情包括科室基本信息、负责人信息和科室医生列表
3. 科室医生展示采用卡片形式，包含头像、姓名、职称和专长等信息
4. 提供预约挂号入口，方便患者直接预约该科室医生
5. 支持查看医生详情，了解更多医生信息

半夜撕代码版权所有

2.6 与其它模块交互

1. **与医生模块的交互**
    - 科室管理模块需要查询医生信息作为科室负责人
    - 科室删除时需要检查科室下是否有医生
    - 前台科室展示需要获取科室下的医生列表
2. **与预约模块的交互**
    - 前台科室展示页面提供预约挂号入口，可直接跳转到预约页面
    - 预约时可按科室筛选医生
3. **与用户权限模块的交互**
    - 科室管理功能仅对管理员开放
    - 前台科室展示功能对所有用户开放

三、总结

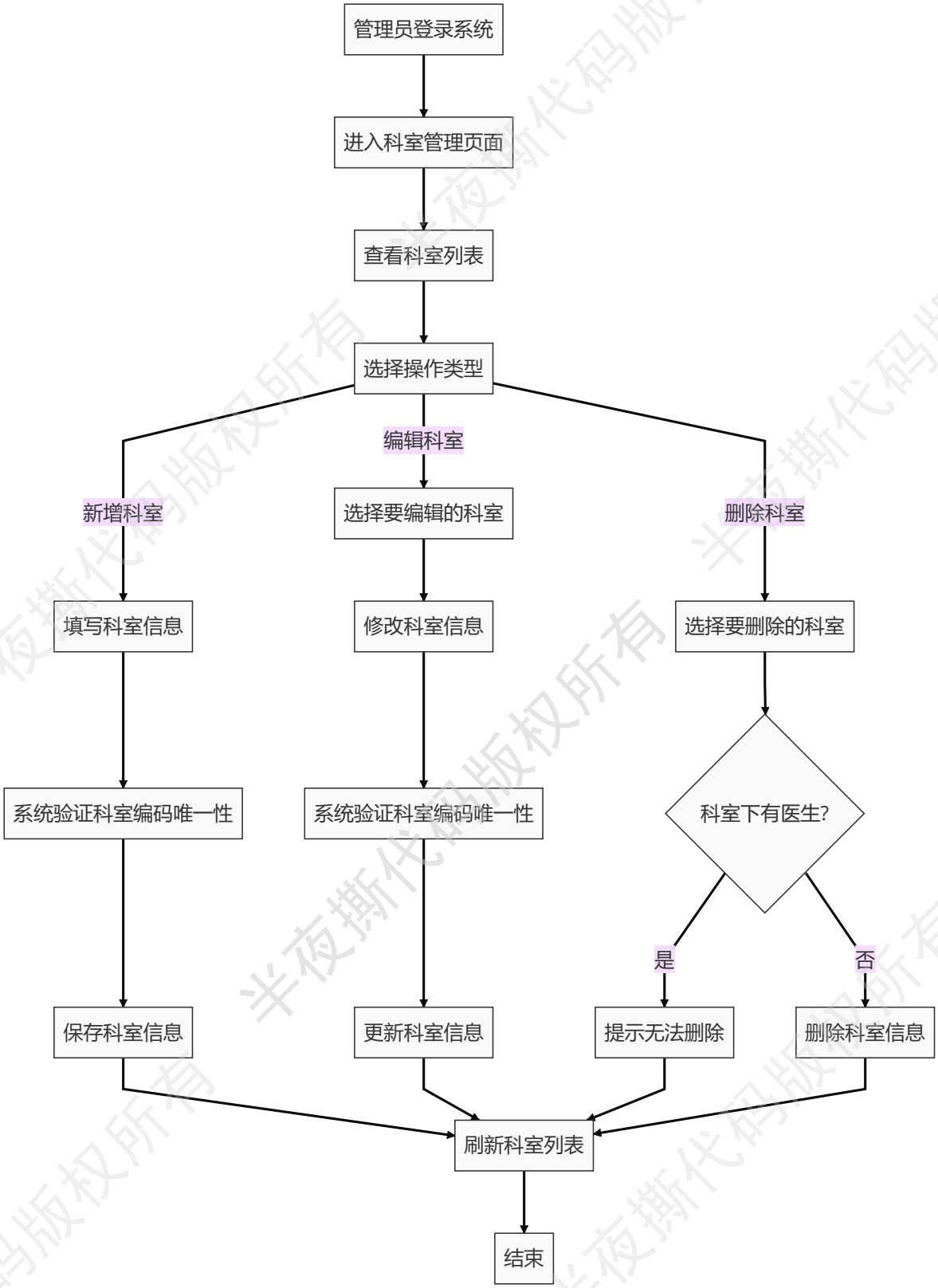
3.1 部门管理功能实现

部门管理模块通过Department实体类实现了科室信息的存储与管理，包含id、deptName、deptCode、description、directorId、status等核心字段。后台管理功能通过DepartmentController提供RESTful API接口，包括新增、编辑、删除、查询等操作。在新增科室时，系统会验证科室编码唯一性并检查负责人是否存在；编辑时会排除自身ID检查编码唯一性；删除时会检查科室下是否有医生，确保数据完整性。前端通过Element Plus组件实现用户界面，支持多条件查询、分页展示和表单验证。

3.2 前台科室展示功能实现

前台科室展示功能为患者提供医院科室信息浏览服务。页面采用左右布局，左侧为科室列表，右侧展示所选科室的详细信息。当用户选择科室时，系统会加载科室详情、负责人信息和科室医生列表。科室医生以卡片形式展示，包含头像、姓名、职称和专长等信息。页面还提供预约挂号入口，方便患者直接预约该科室医生。整个页面设计美观，交互流畅，为患者提供良好的用户体验。

半夜撕代码版权所有


管理员登录系统
进入科室管理页面
查看科室列表
选择操作类型
新增科室
编辑科室
删除科室
填写科室信息
选择要编辑的科室
修改科室信息
选择要删除的科室
系统验证科室编码唯一性
系统验证科室编码唯一性
科室下有医生?
是
否
保存科室信息
更新科室信息
提示无法删除
删除科室信息
刷新科室列表
结束

半夜撕代码版权所有